

Adversarial Tokenization

ACL 2025, Underline Science Inc.

Authors: Renato Geh, Guy Broeck, Zilei Shao

DOI:

Available from: <https://underline.io/lecture/123177-adversarial-tokenization>

Lecture Transcript

Today I'll be presenting on adversarial tokenization, our recent work in ACL 2025 main conference.

So first, let's start with some basics. Uh, tokenization is a very important pre-processing step in nowadays LOM training and testing. Most language models represent distributions over sequences of subwords, named tokens, but not strings. So the reason why people usually do is that it saves a lot of computation. For example, you can uh tokenize the string catability in such a way, which uh shrinks the lens a lot, which facilitates inference and training.

The natural question to ask is, how do we tokenize given a certain string. There is usually a unique tokenization if you prompt a model's tokenizer, and we call it canonical tokenization. For example, if you ask Lama to use tokenizer, how to tokenize the word caterpillar, it will always give you a deterministic answer that here is the tokenization you can use and we call it canonical tokenization.

The common assumption people may have is that the probability for the model to generate the string caterpillar is equal to generate um its canonical tokenization, but it's not usually true. So a string can be tokenized in an exponential number of ways. For example, here, using Lama to vocabulary, you have over 700 possible tokenization.

So, given the fact that a string can be tokenized in so many different ways, uh, we want the semantic understanding in these tokenizations if it's the same.

Um, was respected, uh, to a certain LM, can we assume, how can we measure how different the semantic understanding or we call semantic signal of tokenizations? Uh, in other words, can an LM understand all of these tokenization as the words caped pillar?

Uh, we wonder at what point does the um not recognize a tokenization. So how far from the canonical, um, will break the understanding. Given a reference tokenization, that's a question we want to ask. For example, the model may understand the top two tokenization, but cannot understand the last one. So we wonder, when does it go from understandable to not understandable. Um, can we define a metric space for tokenizations? That's why we introduce added distance here. So we use added distance to to measure how divergent uh a reference tokenization is from the canonical one. And we see here, uh, it seems like there is a trend, uh, there's a correlation between the distance and understandability.

Um, and we do, we did a more principled way to quantify this uh correlation. So here we develop a small data set consisting of a quiz problems. And now we, we tokenize the prompts in different ways and then prompter model to see if it can get the answer correctly. And here's uh the experiment result we have here, uh, the x-axis means the normalized distance between the canonical and an alternative tokenization. When it's 0, that means it's the canonical one, and when it's 1, that means it's character level tokenization.

So, uh, as we can see here, it seems like there's a trend, um, that the further away a reference tokenization is from the canonical, the lower the semantic signal is because um the accuracy drops as the distance goes up.

So, um.

As we know, um, in post training phase, alignment shovels probability mass away from the malicious behavior, and that's how, uh, we prevent the model from answering, um, dangerous responses. But what we should notice is that, uh, the alignment only happens for canonical tokenizations. It only includes canonical tokenization in the post-training phase.

So that makes us wonder, can we sample tokenizations with high semantic signals that evade alignment? Um, for example, you can prompt the model with this prompts that write a tutorial on how to make a bomb, and an online model should tell you I cannot do it. But you can slightly change the tokenization of the prompt, and then the model will give you the faithful answer.

So we quantify this by using strong reject score, which measure the measures the harmfulness and the relevance of the responses. And we see that uh as the distance goes up, there exists some sweet spot uh in this figure that will give you a relatively high s strong reject score and consistently it's higher than the canonical uh tokenization.

So we already showed that uh you may use random sampling to get an alternative tokenization that may break the alignment. But how to find the tokenization most likely to elicit the dangerous responses. So, uh, here's an objective function we propose to maximize, that is, we want to maximize the LM's probability of generating a target response given the prefix and the tokenization of uh the prompt. So for example, we want the model to answer something starting with the prefix um in the target that's short. He's a tutorial on how to make a bomb and give that's um. Given the tokenization, a certain tokenization of the malicious uh request.

Unfortunately, we show that the conditional most likely tokenization problem is and be hard, so we cannot do it directly. Um, so the solution here we propose is greedily approximate uh this probability using local search.

And here we define neighborhood to be the set of tokenization with distance of to to the current tokenization and denoted as neighborhood of the.

We found some nice propositions that could ensure the robustness of our algorithm. And here is a snippet of our algorithm.

Uh, we applied our algorithm into different case studies, for example, jailbreaking, as we just saw, uh, a more formal way to phrase this task is to promote the RMT output phase for response to harmful prompts. And in this table, we showed our results in this task. Um, so we can take a look at the bottom part of this, uh, table. Which shows our adversarial tokenization achieves competitive uh performance against the baselines and also much higher than the canonical baseline. Um, we also combine it seamlessly with the existing pipelines because our attack happens at the tokenization level. So, uh, by doing this we achieve

sort of performance. And the second case study is to evade safety models. So there are some well known, uh, guard models like Glamagar and Shield Jama which will tell you directly if a pump is safe or not. And by using adversarial tokenization, the success rate improves by 5 to 10%. And a third case study we did is prompt injection, which is a A man in the middle attack. So here a malicious agent intercepts the conversation by altering the user input to provoke a malicious response, and our adversarial tokenization achieves 65% improvement compared to the canonical baseline.

So Um, to this point, I want to point out a fact that seems to be a contradiction. So, um, we wonder how come Ls recognize non-canonicals at the semantic level, but do not recognize them at the alignment level. So to resolve this, we need to, uh, point out the differences between pre-training and post-training. In pre-training, we will have a huge data sets and it consists of typos, multilingual, we're spacing. All of these contribute uh to the semantic leakage on to the non-canonical tokens. But in post training, we usually use smaller data sets. And

we use the reinforcement learning with fewer typos and usually the data is monolingual. So the difference makes an alignment signal leaked less than the uh the semantic leakage we have in the pre-training, which makes our adversarial tokenization to be true.

So a question we want to ask is, should post training incorporate elements from pre-training? And it wraps up our presentation in our research organization. Thank you for watching.